

MEMORY MANAGEMENT

To store anything in our computer, we should have to allocate the memory first.

This memory allocation is conducted in two ways.

1. Static memory allocation.
2. Dynamic memory allocation.

In static memory allocation, the memory specified at compile/design time, based on the data type or array size. This type of memory management is called compile time memory management [**compiler indicates memory and O.S allocates the memory**].

In static memory allocation, the memory size is fixed at compile time and we can't

change this memory size at run time. It causes some times memory wastage / shortage.

To avoid this problem, the only solution is dynamic memory allocation.

In dynamic memory allocation, the memory is allocated at run time, based on the user input, instantly.

This type of memory management is called run time memory management.

To conduct dynamic memory allocation, we should have to use **pointers**.

In dynamic memory allocation the memory is allocated in **HEAP** area.

To manage the dynamic memory, we are using some predefined functions like

- malloc()
- calloc()
- realloc()
- free()

All these functions are available in **<alloc.h>**

malloc(), realloc(), calloc() functions are able to allocate the memory of **64KB**

Maximum at a time.

To allocate more than 64KB memory, use the functions

- farmalloc()
- farcalloc()
- farrealloc().

Note:

when we are working with dynamic memory allocation, we have to allocate the

memory for any data type. Due to this all these functions return datatype is **void ***, which is a generic type. Due to this we should have to provide **explicit type casting** for all these functions.

malloc()	calloc()
Block Memory allocation	Contiguous memory blocks allocation
Allocates memory in bytes form	Allocates memory in blocks form.
Initial values garbage	Initial values 0
One argument required	Two arguments required
Used for normal variables	Used for array type variables

Syntax:

```
void * malloc(bytes);
```

```
void * calloc(no of blocks, block_size);
```

free(): It is used to release the memory allocated by malloc(), calloc() and realloc().

Syntax: void free(pointer);

realloc(): It is used to extend the memory allocated by malloc() or calloc() at runtime. Working style is similar to malloc().

Syntax: void * realloc(oldptr, newsize);

free example:

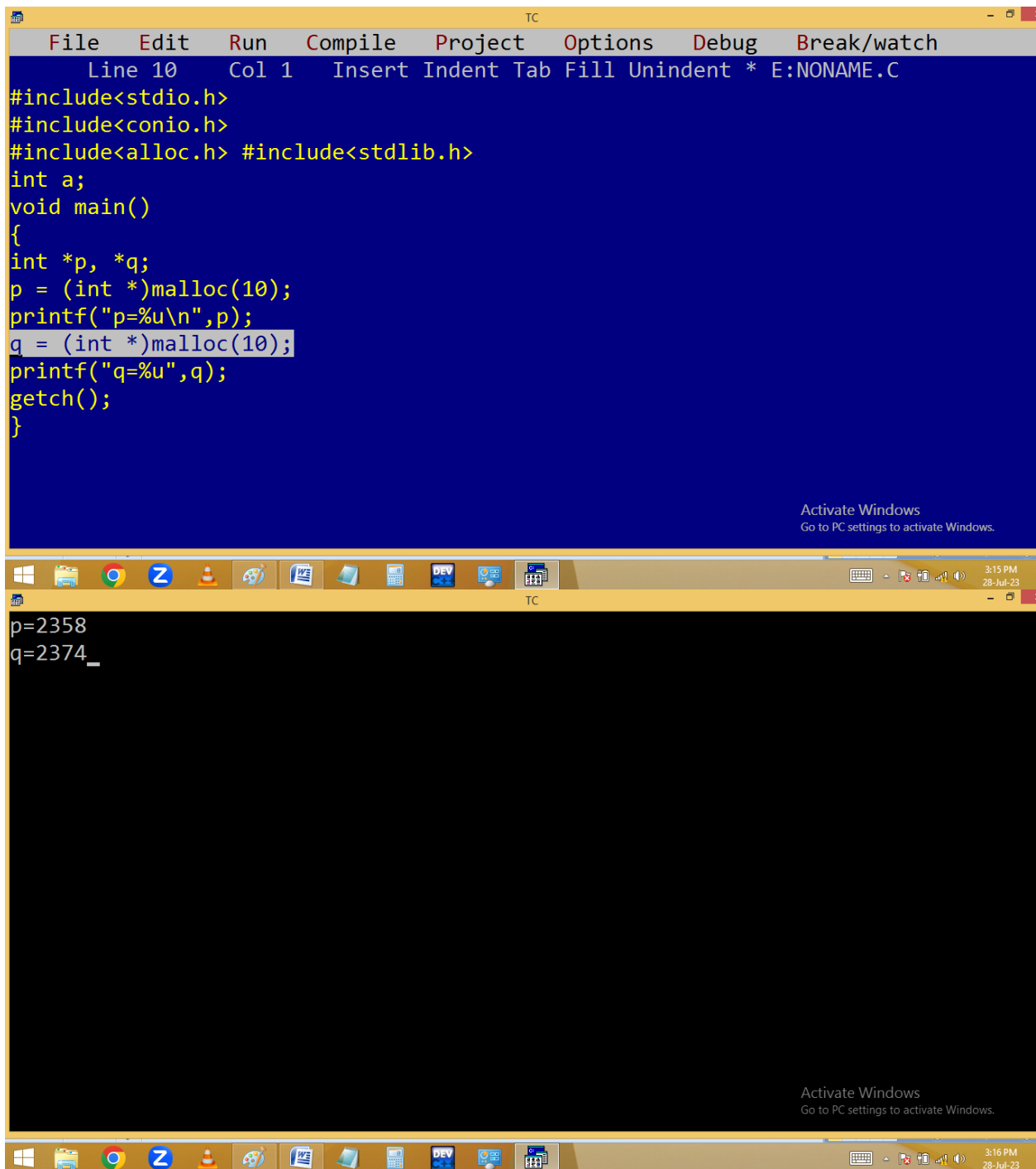
The image shows a Turbo C++ (TC) IDE window. The top menu bar includes File, Edit, Run, Compile, Project, Options, Debug, and Break/watch. The status bar at the top indicates 'Line 10 Col 9 Insert Indent Tab Fill Unindent * E:NONAME.C'. The code editor contains the following C program:

```
#include<stdio.h>
#include<conio.h>
#include<alloc.h> #include<stdlib.h>
int a;
void main()
{
    int *p, *q;
    p = (int *)malloc(10);
    printf("p=%u\n",p);
    free(p);
    q = (int *)malloc(10);
    printf("q=%u",q);
    getch();
}
```

The output window at the bottom shows the execution results:

```
p=2358
q=2358
```

Both the code editor and the output window display an 'Activate Windows' watermark in the bottom right corner, stating 'Go to PC settings to activate Windows.' The Windows taskbar at the bottom shows the time as 3:15 PM on 28-Jul-23.



The image shows a screenshot of the Turbo C++ (TC) IDE. The top window displays a C program with the following code:

```
File Edit Run Compile Project Options Debug Break/watch
Line 10 Col 1 Insert Indent Tab Fill Unindent * E:NONAME.C
#include<stdio.h>
#include<conio.h>
#include<alloc.h> #include<stdlib.h>
int a;
void main()
{
int *p, *q;
p = (int *)malloc(10);
printf("p=%u\n",p);
q = (int *)malloc(10);
printf("q=%u",q);
getch();
}
```

The bottom window shows the output of the program:

```
p=2358
q=2374_
```

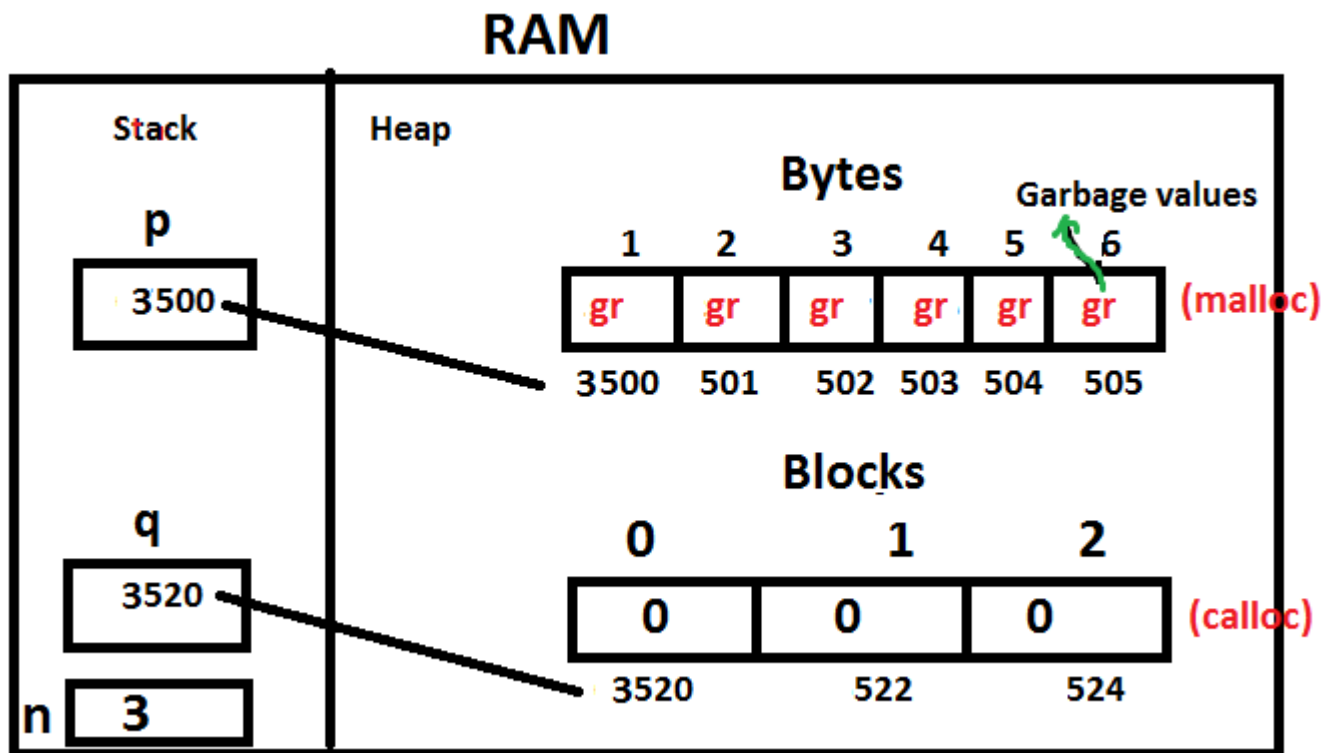
Both windows include a status bar at the bottom with the text "Activate Windows Go to PC settings to activate Windows." and a taskbar at the very bottom showing various application icons and the system clock (3:15 PM and 3:16 PM on 28-Jul-23).

allocating memory for 3 integers using
malloc(), calloc().

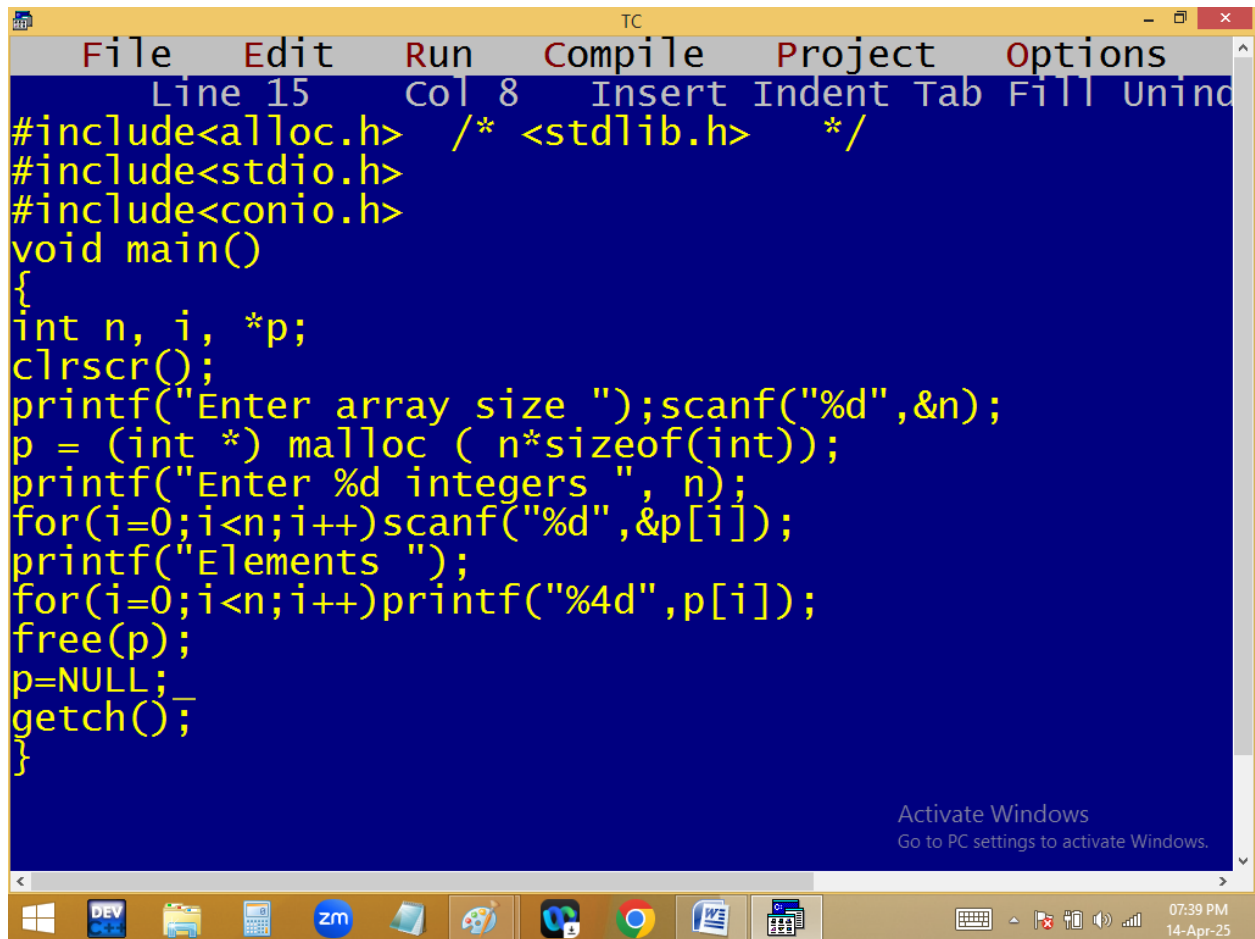
```
int *p, *q, n=3;
```

```
p = (int *)malloc(n * sizeof(int));
```

```
q = (int *)calloc(n , sizeof(int));
```



creating a dynamic one dimensional array:



```
TC
File Edit Run Compile Project Options
Line 15 Col 8 Insert Indent Tab Fill Undo
#include<alloc.h> /* <stdlib.h> */
#include<stdio.h>
#include<conio.h>
void main()
{
int n, i, *p;
clrscr();
printf("Enter array size ");scanf("%d",&n);
p = (int *) malloc ( n*sizeof(int));
printf("Enter %d integers ", n);
for(i=0;i<n;i++)scanf("%d",&p[i]);
printf("Elements ");
for(i=0;i<n;i++)printf("%4d",p[i]);
free(p);
p=NULL;
getch();
}
```

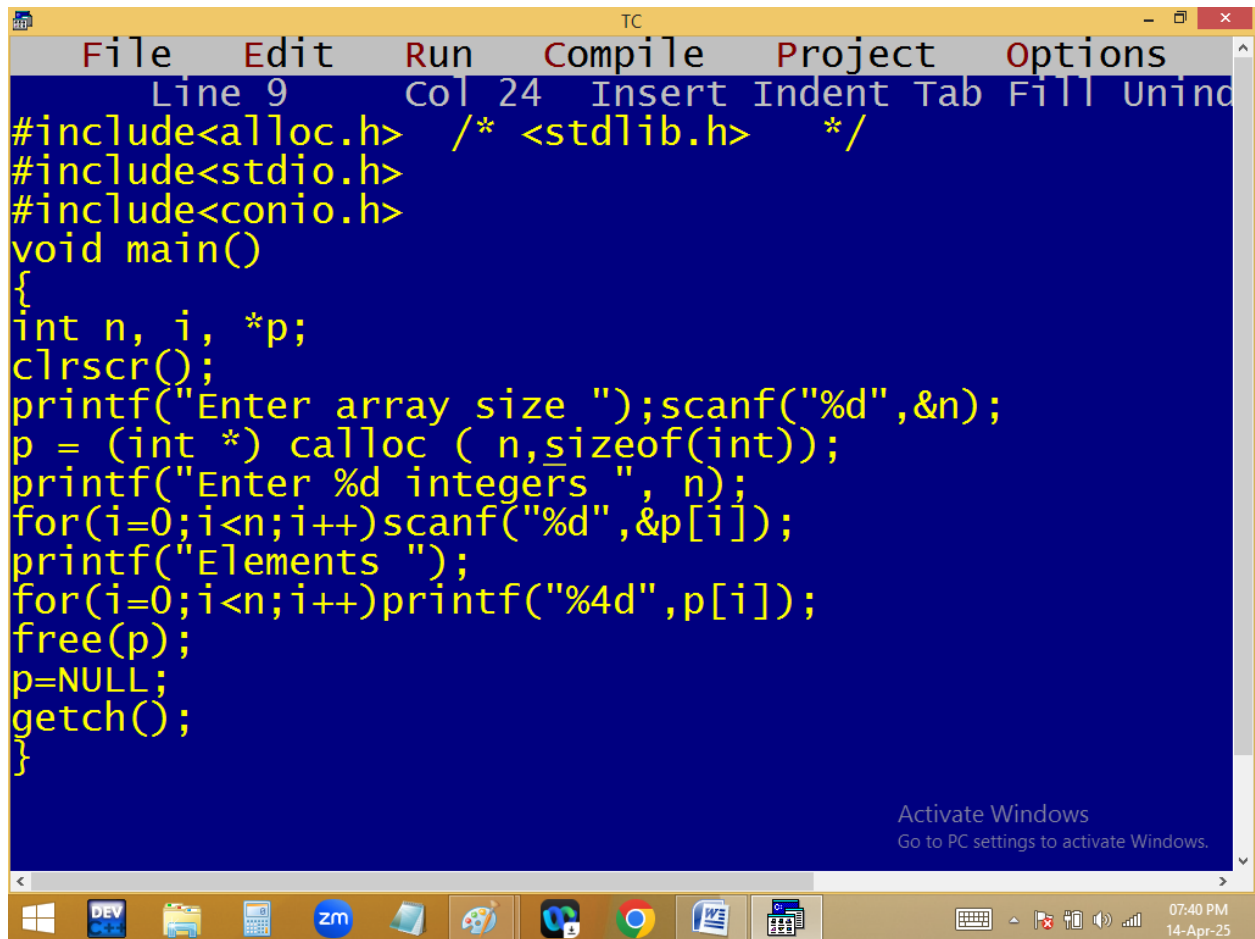
Activate Windows
Go to PC settings to activate Windows.

07:39 PM
14-Apr-25

```
TC
Enter array size 7
Enter 7 integers 1 0 2 7 3 6 4
Elements      1    0    2    7    3    6    4
```

Activate Windows
Go to PC settings to activate Windows.

07:39 PM
14-Apr-25



The image shows a screenshot of a Turbo C++ (TC) IDE window. The title bar reads "TC". The menu bar includes "File", "Edit", "Run", "Compile", "Project", and "Options". The status bar at the top indicates "Line 9", "Col 24", and "Insert" mode. The main editing area has a blue background with yellow text. It contains a C program that dynamically allocates an array using `calloc` and frees it using `free`. The program prompts the user to enter an array size and then integers, displaying the entered elements. The Windows taskbar is visible at the bottom, showing various application icons and the system clock.

```
File Edit Run Compile Project Options
Line 9 Col 24 Insert Indent Tab Fill Unind
#include<alloc.h> /* <stdlib.h> */
#include<stdio.h>
#include<conio.h>
void main()
{
int n, i, *p;
clrscr();
printf("Enter array size ");scanf("%d",&n);
p = (int *) calloc ( n,sizeof(int));
printf("Enter %d integers ", n);
for(i=0;i<n;i++)scanf("%d",&p[i]);
printf("Elements ");
for(i=0;i<n;i++)printf("%4d",p[i]);
free(p);
p=NULL;
getch();
}
```

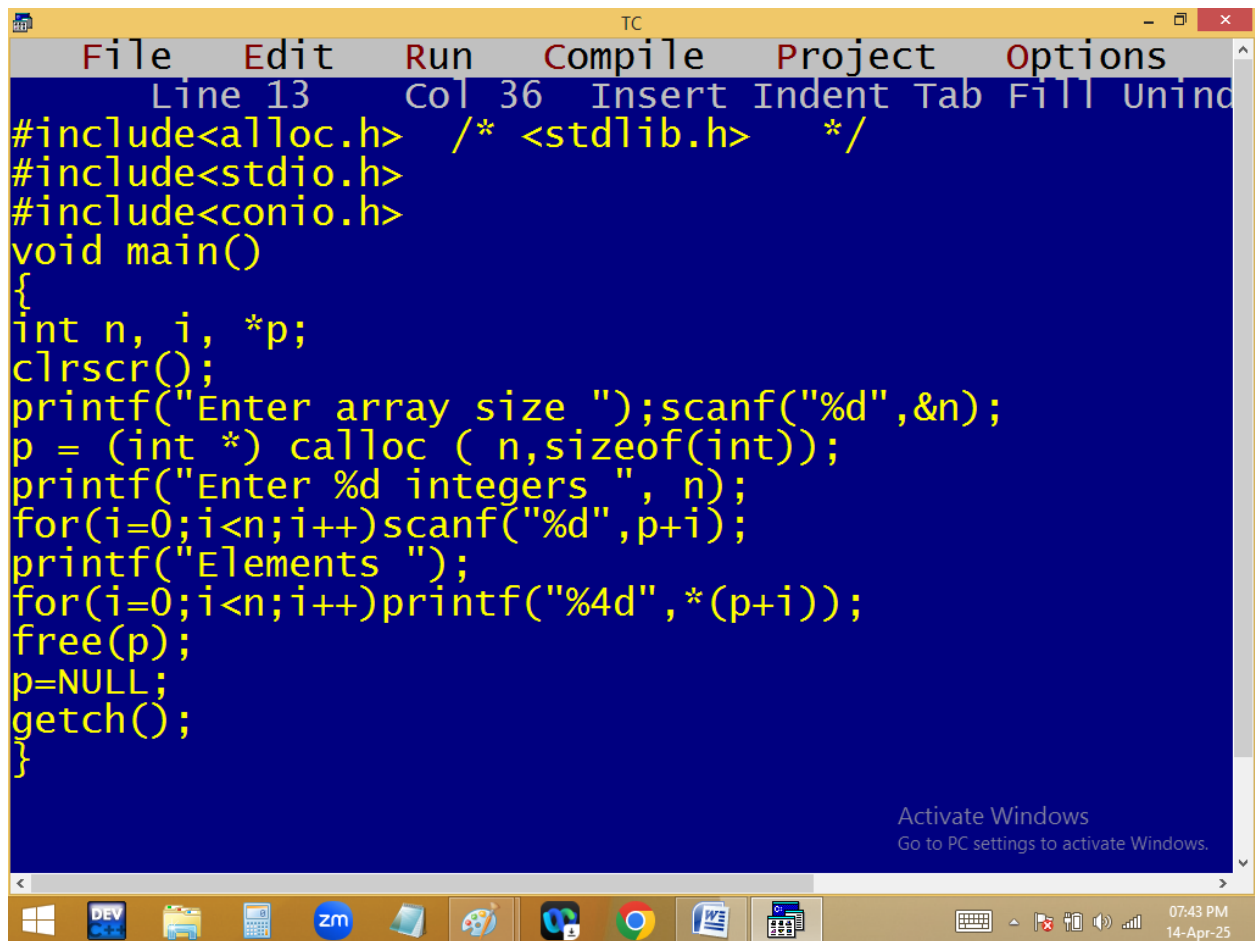
Activate Windows
Go to PC settings to activate Windows.

07:40 PM
14-Apr-25

```
TC
Enter array size 4
Enter 4 integers 1 2 3 4
Elements      1      2      3      4_

Activate Windows
Go to PC settings to activate Windows.
```





The image shows a screenshot of a Turbo C++ (TC) IDE window. The title bar reads "TC". The menu bar includes "File", "Edit", "Run", "Compile", "Project", and "Options". The status bar at the top indicates "Line 13 Col 36 Insert Indent Tab Fill Unind". The main editing area has a blue background and contains the following C code:

```
#include<alloc.h> /* <stdlib.h> */
#include<stdio.h>
#include<conio.h>
void main()
{
int n, i, *p;
clrscr();
printf("Enter array size ");scanf("%d",&n);
p = (int *) calloc ( n,sizeof(int));
printf("Enter %d integers ", n);
for(i=0;i<n;i++)scanf("%d",p+i);
printf("Elements ");
for(i=0;i<n;i++)printf("%4d",*(p+i));
free(p);
p=NULL;
getch();
}
```

An "Activate Windows" watermark is visible in the bottom right corner of the IDE window, stating "Go to PC settings to activate Windows." The Windows taskbar is at the bottom, showing icons for Windows, DEV, File Explorer, Calculator, Zoom, and several other applications. The system clock in the bottom right corner shows "07:43 PM 14-Apr-25".

```
TC
Enter array size 5
Enter 5 integers 2 0 1 3 1
Elements      2    0    1    3    1_

Activate Windows
Go to PC settings to activate Windows.
```

